Goroutines et Channels

---

1. Concurrence vs Parallélisme et l'Ordonnanceur Go (`G-M-P`)

1.1 La distinction fondamentale

- **La Concurrence (*Concurrency*)** est une propriété d'**architecture logicielle** : c'est la capacité d'un programme à décomposer et gérer plusieurs tâches indépendantes en même temps (même sur un processeur à 1 seul cœur par entrelacement temporel).
- **Le Parallélisme (*Parallelism*)** est une propriété d'**exécution matérielle** : c'est le fait d'exécuter physiquement plusieurs calculs au même instant précis sur plusieurs cœurs CPU distincts.

1.2 Pourquoi les Threads OS (`pthread` en C / Threads Java) passent mal à l'échelle ?

Un thread du système d'exploitation (Linux/Windows) est lourd :

1. **Empreinte mémoire fixe importante :** Chaque thread OS réserve par défaut une pile (*Stack*) de **1 Mo à 8 Mo** en mémoire virtuelle (avec une page de garde). Lancer 100 000 threads réclame plus de 100 Go de RAM !
2. **Changement de contexte coûteux (*Context Switch*) :** Passer d'un thread OS à un autre nécessite une interruption matérielle (passage en mode Noyau *Ring 0*), la sauvegarde de dizaines de registres CPU et l'invalidation des caches processeur (~1 à 2 microsecondes).

1.3 Les Goroutines et le modèle d'ordonnancement `M:N` (`G-M-P`)

En Go, on ne manipule jamais directement les threads OS. On lance des **Goroutines** avec le mot-clé `go` :

- **Empreinte mémoire minuscule :** Une goroutine démarre avec une pile de seulement **2 Ko** (allouée sur le tas et gérée par le runtime Go). Cette pile peut grandir et rétrécir dynamiquement jusqu'à 1 Go si nécessaire !
- **Coût de création dérisoire :** On peut faire tourner **1 million de goroutines simultanées** sur un simple ordinateur portable (environ 2 Go de RAM consommés).

Le runtime Go intègre son propre ordonnanceur coopératif et préemptif en espace utilisateur (*User-space Scheduler*), basé sur l'architecture **G-M-P** :

1. **`G` (*Goroutine*) :** La tâche à exécuter (son code, sa pile de 2 Ko, son pointeur d'instruction).
2. **`M` (*Machine*) :** Un véritable Thread du système d'exploitation géré par l'OS.
3. **`P` (*Processor*) :** Un contexte logique d'exécution (possédant une file d'attente locale de Goroutines prêtes à tourner). Par défaut, le nombre de `P` est égal au nombre de cœurs logiques de la machine (`runtime.GOMAXPROCS(0)`).

Magie de l'ordonnanceur Go (*Work Stealing* & I/O non bloquant) :

- Lorsqu'une goroutine `G1` fait un appel réseau ou lit un fichier, elle ne bloque **pas** le thread OS `M` : le runtime Go détache `G1` (confiée au *Netpoller* asynchrone `epoll`/`kqueue`) et exécute immédiatement la goroutine `G2` sur le même thread CPU !
- Si un processeur logique `P1` n'a plus rien dans sa file locale, il **vole la moitié des goroutines** de la file d'attente d'un autre processeur `P2` (*Work Stealing*).

---

2. Lancer et comprendre le cycle de vie des Goroutines

On lance n'importe quel appel de fonction dans une nouvelle goroutine en le précédant simplement du mot-clé **`go`** :

```go
Go
package main

import (
	"fmt"
	"time"
)

func traiterTache(id int) {
	fmt.Printf("[Worker %d] Démarrage du traitement...\n", id)
	time.Sleep(100 * time.Millisecond)
	fmt.Printf("[Worker %d] Terminé !\n", id)
}

func main() {
	// Lancement de 3 goroutines concurrentes (asynchrone, non bloquant)
	for i := 1; i <= 3; i++ {
		go traiterTache(i)
	}

	// La fonction main() tourne elle-même dans la "goroutine principale".
```

```
	// Si main() se termine, le processus entier s'arrête et tue
	// toutes les autres goroutines en cours d'exécution
	time.Sleep(200 * time.Millisecond) // Attente (Channels ou WaitGroup !)
}
```

2.1 Note historique importante : Capture de variable de boucle (`Go 1.22+`)

Dans une boucle `for i := 1; i <= 3; i++` lançant une closure `go func() { fmt.Println(i) }()` :

- **Avant Go 1.22 :** La variable `i` était partagée entre toutes les itérations. Le temps que les goroutines démarrent, la boucle était finie et les 3 goroutines affichaient `4, 4, 4` ! Il fallait passer `i` en paramètre de la fonction anonyme `go func(val int) { ... }(i)`.
- **Depuis Go 1.22 :** Le compilateur crée une nouvelle instance de la variable de boucle à chaque itération. Cependant, passer explicitement les arguments à la fonction lancée avec `go` reste une excellente pratique de clarté.

---

3. Les Channels (`chan T`) : Le modèle CSP

En C, C++ ou Java, plusieurs threads communiquent en lisant et écrivant dans les mêmes zones mémoire partagées en les protégeant par des verrous (`mutex`). En Go, la philosophie issue du modèle **CSP** (*Communicating Sequential Processes* de Tony Hoare) est résumée par :

> ***"Do not communicate by sharing memory; instead, share memory by communicating."***

Un **Channel** est un tuyau de communication **typé** et **Thread-Safe** (sécurisé par construction) permettant de faire passer des données d'une goroutine à une autre tout en les synchronisant.

3.1 Opérations de base et Channels non-bufferisés (Synchrones)

Par défaut, `make(chan T)` (sans second argument) crée un **channel non-bufferisé** (capacité = `0`), aussi appelé *Rendez-vous Channel* :

```
Go
ch := make(chan string) // Channel synchrone de capacité 0

// 1. Envoyer une valeur dans le channel : ch <- valeur
// BLOQUANT tant qu'aucune autre goroutine n'est prête à lire sur 'ch' au même instant !
```

```
// 2. Recevoir une valeur depuis le channel : variable := <-ch
// BLOQUANT tant qu'aucune autre goroutine n'a envoyé de valeur dans 'ch' !
```

Exemple de synchronisation parfaite sans `time.Sleep` :

```
Go
func main() {
	resultatChan := make(chan int) // Capacité 0

	go func() {
		// Calcul lourd dans une goroutine séparée
		somme := 21 * 2
		resultatChan <- somme // Bloque jusqu'à ce que main() lise !
	}()

	// main() se met en pause sur '<-resultatChan' jusqu'à l'arrivée de la
donnée :
	valeur := <-resultatChan
	fmt.Println("Résultat reçu :", valeur)
}
```

3.2 Channels Bufferisés (Asynchrones jusqu'à saturation)

En passant une capacité $N > 0$ à `make(chan T, N)`, on crée un channel doté d'une file d'attente interne (FIFO) de N places :

- **L'envoi `ch <- v` est NON-BLOQUANT** tant que le tampon n'est pas plein (`len(ch) < cap(ch)`). Dès que le tampon est plein (`len == cap`), l'émetteur bloque jusqu'à ce qu'un lecteur libère une place.
- **La réception `<-ch` est NON-BLOQUANTE** tant qu'il reste au moins un élément dans le tampon (`len(ch) > 0`). S'il est vide, le lecteur bloque.

```
Go
fileTaches := make(chan string, 2) // Tampon de 2 places

fileTaches <- "Tâche A" // Non bloquant (1/2)
fileTaches <- "Tâche B" // Non bloquant (2/2 - Plein !)
// fileTaches <- "Tâche C" // BLOQUERAIT ici (Deadlock si aucune autre goroutine
ne lit !)

fmt.Println(<-fileTaches) // "Tâche A" (libère 1 place dans le tampon)
```

---

4. Directionnalité, Fermeture (`close`) et Matrice de comportement

4.1 Channels directionnels (Sécurité à la compilation)

Dans la signature d'une fonction, il est vivement recommandé de restreindre le sens d'utilisation d'un channel pour que le compilateur interdise toute mauvaise manipulation (notamment fermer un channel du mauvais côté) :

- **`ch chan<- int` (Send-Only) :** On peut uniquement envoyer dedans (`ch <- 42`). Tenter de lire `<-ch` est une erreur de compilation.
- **`ch <-chan int` (Receive-Only) :** On peut uniquement lire dessus (`v := <-ch`). Tenter d'écrire dedans ou d'appeler `close(ch)` est une erreur de compilation !

(Un channel bidirectionnel `chan int` créé avec `make` est implicitement converti par Go en `chan<- int` ou `<-chan int` lors de l'appel de fonction).

4.2 Fermeture d'un channel (`close`) et boucle `for range`

Lorsqu'un producteur a terminé d'envoyer toutes ses données, il peut fermer le channel avec `close(ch)` pour prévenir le(s) consommateur(s) qu'aucune nouvelle donnée n'arrivera plus :

```Go
// Producteur : a le droit d'écrire et de fermer (chan<- int)
func genererNombres(out chan<- int) {
	for i := 1; i <= 5; i++ {
		out <- i
	}
	close(out) // Signal de fin de flux (Toujours fermé par l'ÉMETTEUR, jamais par le lecteur !)
}

func main() {
	flux := make(chan int, 2)
	go genererNombres(flux)

	// La boucle 'for ... range' sur un channel lit les valeurs au fur et à mesure
	// et S'ARRÊTE AUTOMATIQUEMENT dès que le channel est fermé ET vidé !
	for n := range flux {
		fmt.Println("Reçu :", n)
	}
	fmt.Println("Flux terminé proprement.")
}
```

Tester manuellement si un channel est fermé : `val, ouvert := <-ch`. Si `ouvert` vaut `false`, le channel est fermé et vidé, et `val` contient la *Zero Value* du type.

4.3 La Matrice des Channels

Le comportement d'un channel dépend de son état (**`nil`**, **Ouvert**, ou **Fermé**). Toute erreur ici entraîne soit un blocage infini (*Deadlock*), soit un crash (*Panic*) :

Opération	**Channel `nil` (`var c chan int`)**	**Channel Ouvert (`make(chan int)`)**	**Channel Fermé (`close(c)`)**
Lire (`<-c`)	**BLOQUE À L'INFINI**	Lit ou attend une valeur	**Ne bloque jamais** : renvoie immédiatement la *Zero Value* (et `ok = false`)
Écrire (`c <- v`)	**BLOQUE À L'INFINI**	Écrit ou attend une place	**`PANIC` IMMÉDIAT !** (`send on closed channel`)
Fermer (`close(c)`)	**`PANIC` IMMÉDIAT !**	Ferme le channel	**`PANIC` IMMÉDIAT !** (`close of closed channel`)
